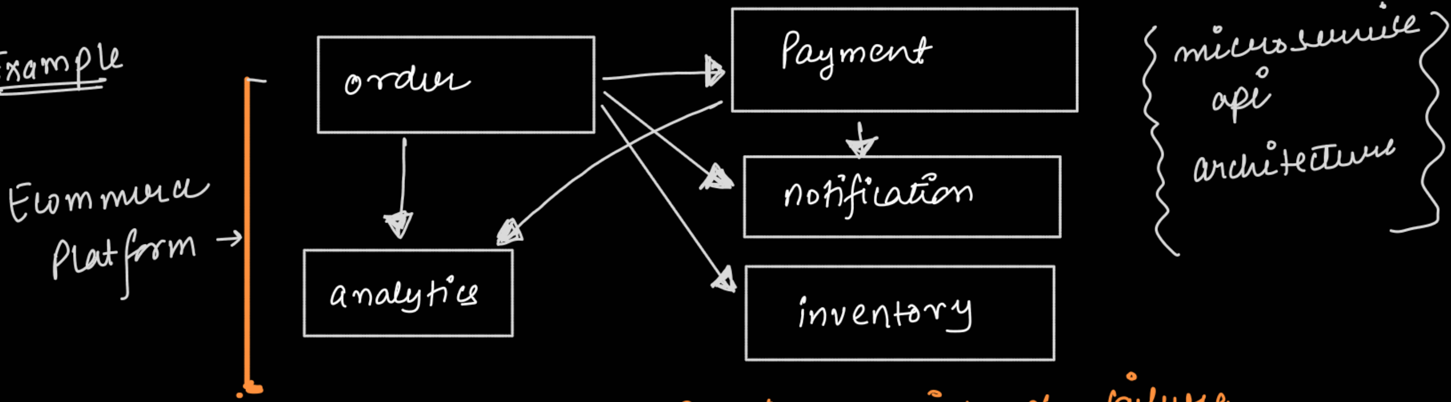
Apache Kafka

→ Distributed System

→ Streaming data platform

Example



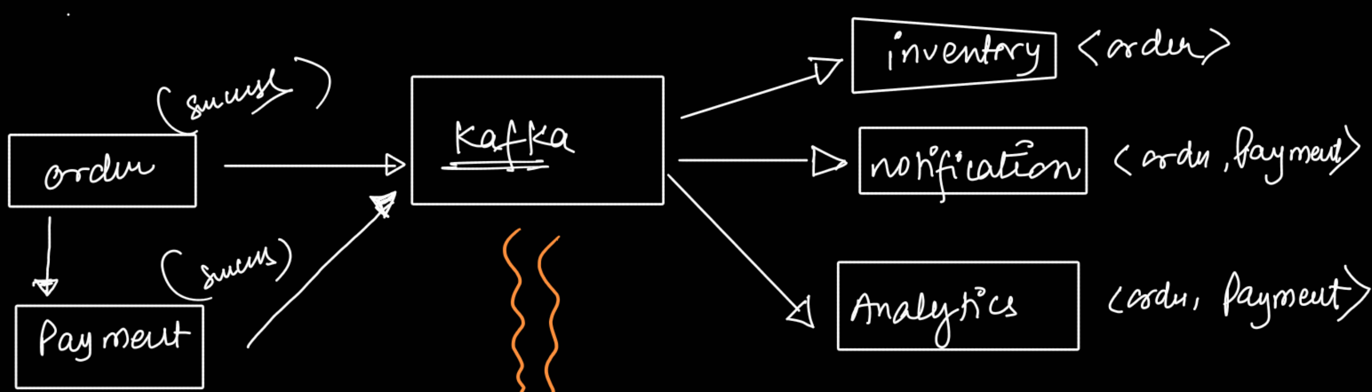
① tight coupling

② Service operates in linear manner

③ single point of failure

④ Losing Data

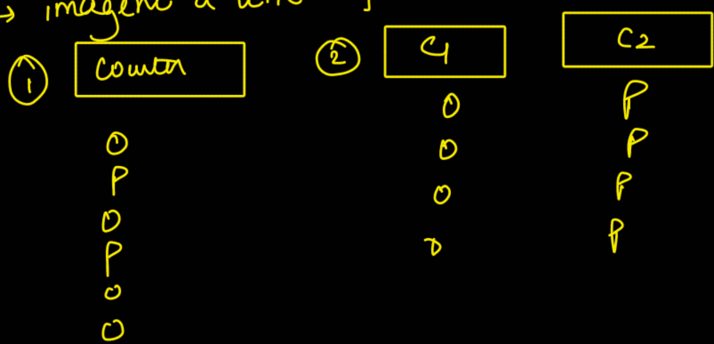
Approach with publish subscribe model



→ 1 Bucket will have lot of data

→ hard to manage

→ imagine a line of Bank.



④ Kafka vs Database

Kafka

→ Real time analytics

Database

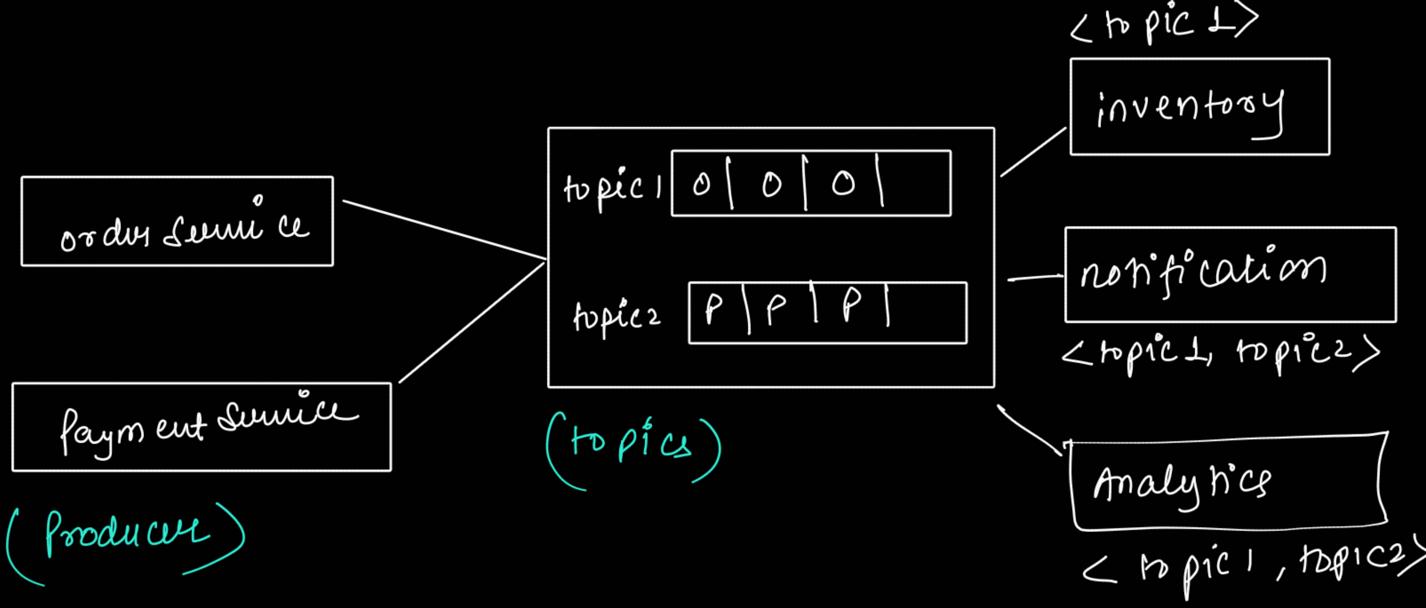
→ Permanent storage of Data

→ Driver location updates

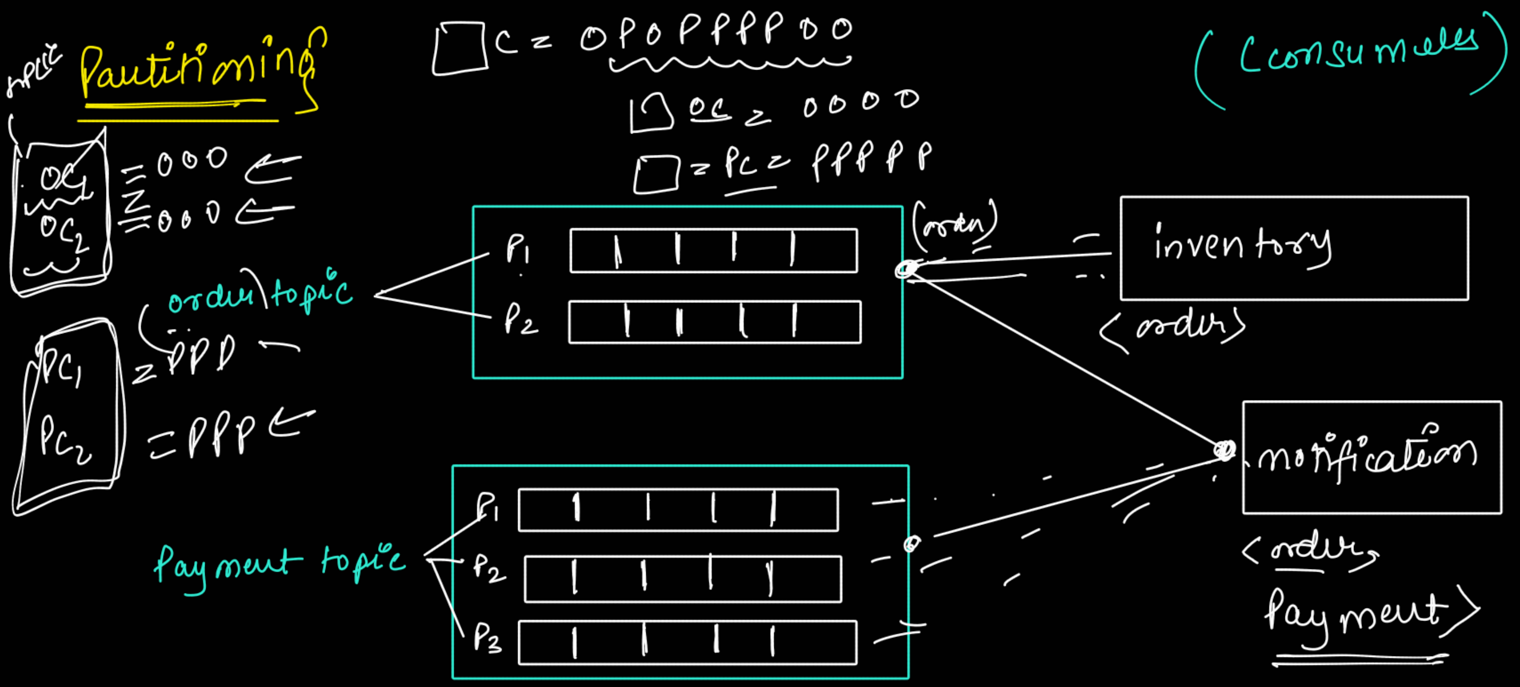
→ streaming analytics Dashboard

Fundamental

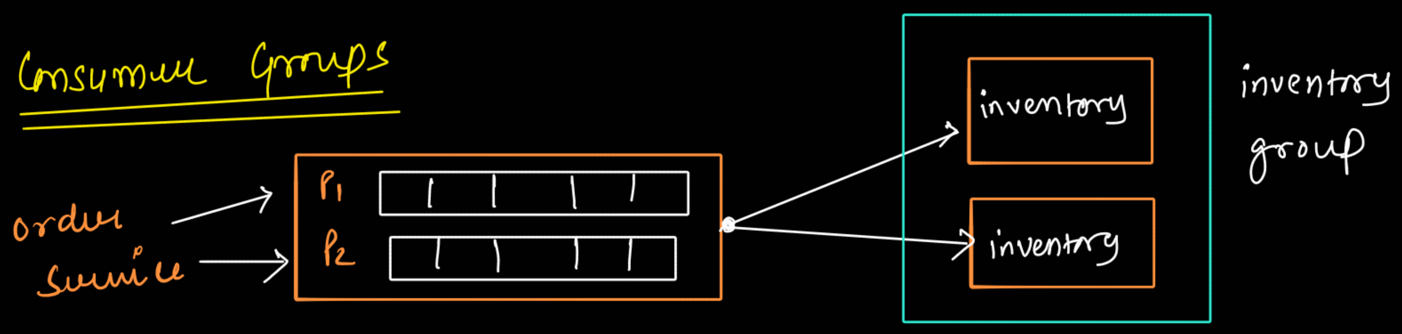
① Producer ② Consumer ③ Topics

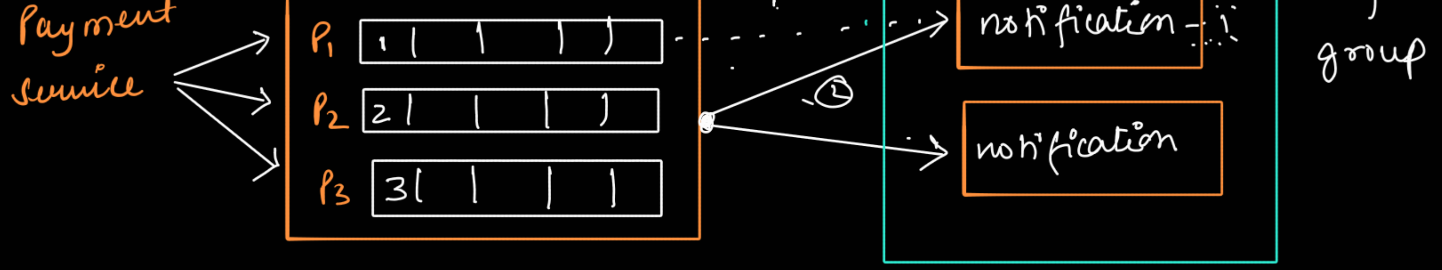


Partitioning



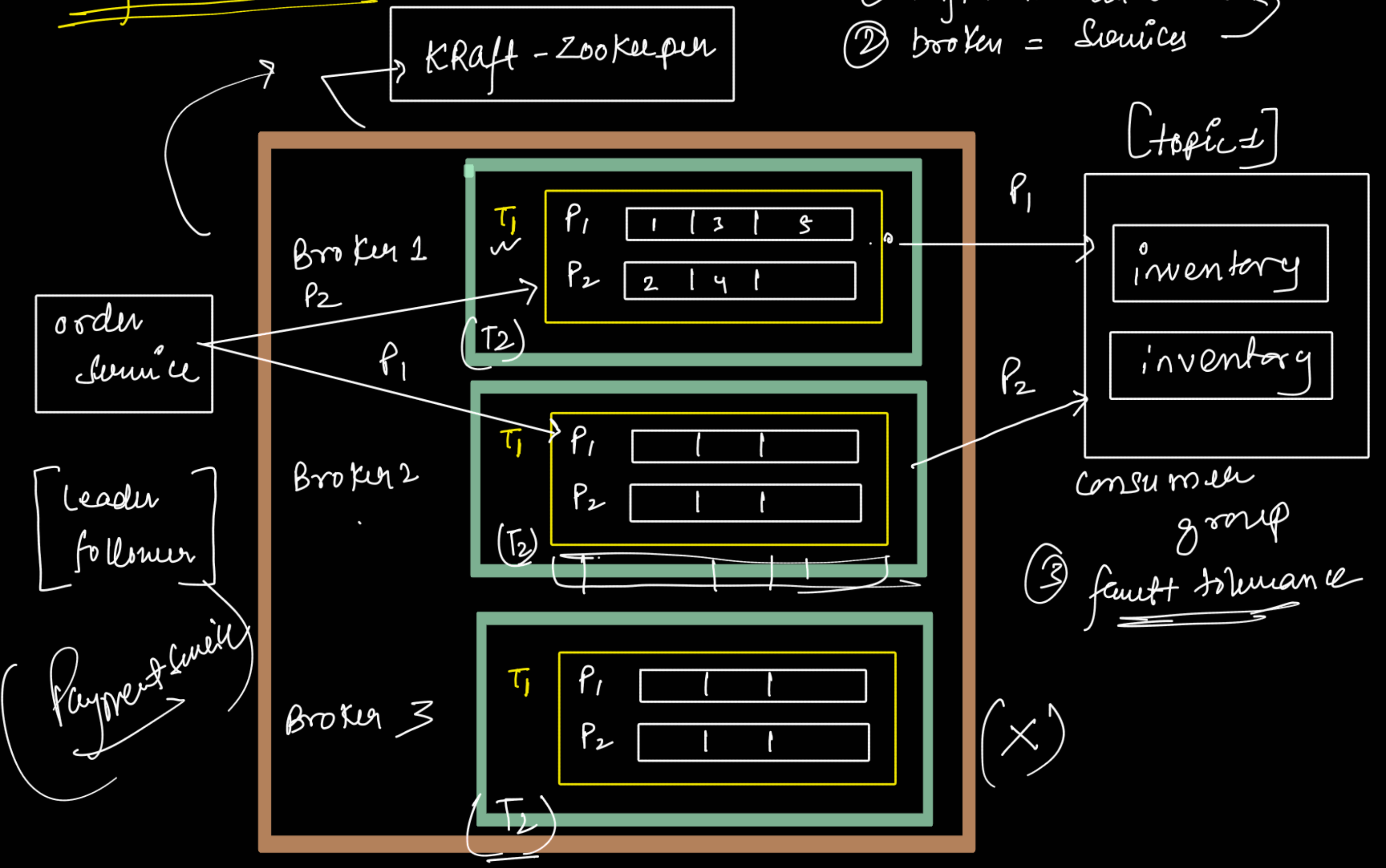
Consumer Groups





Kafka Broker

1. Kafka is clustered
2. broker = services



3. fault tolerance

Kafka vs sqs

1. Pub Sub
2. Retention Policy
3. Kafka

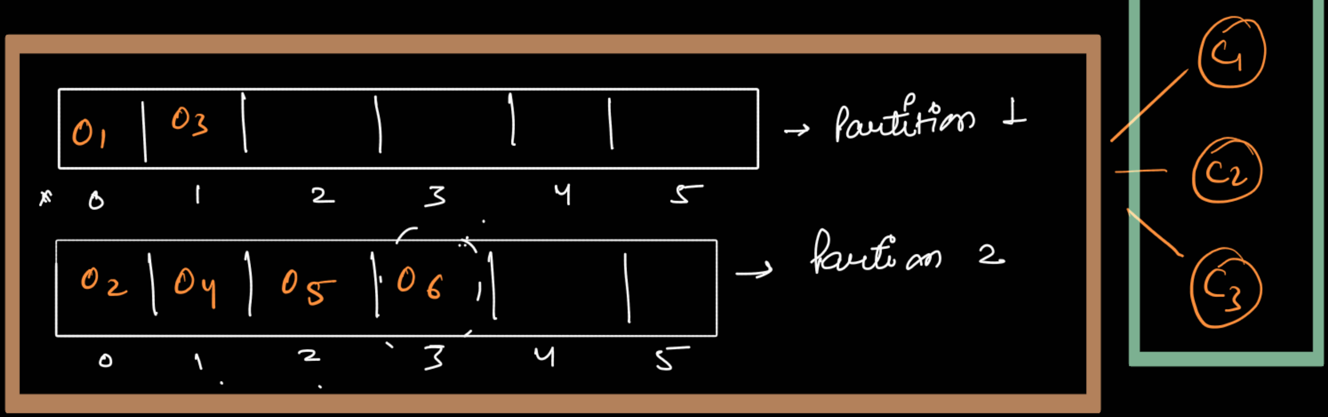
1. queue
2. no retention policy
3. Amazon sqs

retention.ms = -1

retention.ms = 604800000	→ 7 days

Offset

topic



C_3 → idle ×

__consumer_offsets: <internal topic>

Group name	topic	Partition	offset
order-consumer	order	1	1
order consumer	order	2	3

1. Track what has been consumed
2. Avoid message duplication
3. Replay message
4. Fault tolerance
5. order guarantee

Interview questions

1. Load 1 million/sec
   3 topic [order, payment, notification]
   Question → how many partition, how to decide setup, scaling etc?

Answer
- → throughput per partition
- → consumer scaling
- → fault tolerance / replication

Step 1:

1. 1000000/3 = 333000 message/sec
2. kafka → [100K - 200K] message/sec per partition
   ↳ Benchmarking according to production hardware
3. 333K/150K = [2.2] → [for convience → 3, for safety → 4] → partition each topic

order topic → 333K m/s → 4 Partition

payment topic → 333K m/s → 4 Partition

notification topic → 333K m/s → 4 Partition

(4) Replication-factor $\boxed{3}$ → standard production setup.
→ Survive 2 broker failure

(5) Smart Broker-topic-Partition Distribution

each topic → 333K m/s

Broker 1 → $\boxed{t_{10}, t_{11}, t_{20}, t_{30}}$
→ follower

t_1 — 0, 1, 2, 3

Broker 2 → $\boxed{t_{12}, t_{21}, t_{22}, t_{31}}$
→ follower

t_2 — 0, 1, 2, 3

Broker 3 → $\boxed{t_{13}, t_{23}, t_{32}, t_{33}}$
→ follower

t_3 — 0, 1, 2, 3

(3) [no. of consumer usually should be equal to partition]

→ for each topic → 4 → partition → 4 consumers
↳ [in 1 consumer group]

P_1	C_1
P_2	C_2
P_3	C_3
P_4	C_4

✓

P_1	C_1
P_2	C_2
P_3, P_4	C_3

✗

P_1	C_1
P_2	C_2
P_3	C_3
P_4	C_4
	(C_5)

✗

(4) Each message size

333K msg/sec × 1 Kb/message × 86400sec/day

= 28 TB/day/Topic = 3 × 28 Tb/day

= 54 Tb/day

→ So we need lot of storage

→ if you can afford { → 7 days
→ 1 day }

Interview Question

order → 1 million msg/sec

Payment → 200K msg/sec

notification → 100 K msg/sec

average → [Kafka → 150 K msg/sec →]

12 partition

12 × 3 = 36 partitions

→ (1 million)

→ 1.Jay

order → 7 partition

notification - 1

payments - 2

10 → replication = 3 = 10 × 3 = (30 partition)

[if we take 3 broker = 10 partitions]

[if we take 5 broker = 6 partition]

→ future growth

→ Each broker handle

→ 200k msg/sec